

SharksML

June 16, 2026

```
[1]: """
    Shark Attack Severity Predictor - Random Forest Model
    Dataset: California Shark Incidents 1950-2022
    Target: Predict injury severity (Minor / Major / Fatal)
    """

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score, \
    StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)
import warnings
warnings.filterwarnings("ignore")

[2]: PALETTE = {"Fatal": "#c0392b", "Major": "#e67e22", "Minor": "#27ae60"}
COLORS = ["#2c3e50", "#2980b9", "#27ae60", "#e67e22", "#c0392b", "#8e44ad"]
plt.rcParams.update({
    "figure.facecolor": "#f9f9f9",
    "axes.facecolor": "#ffffff",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "DejaVu Sans",
})

[4]: df = pd.read_csv("SharkIncidents_1950_2022.csv")

[5]: # Date features
df["Date"] = pd.to_datetime(df["Date"], format="%m/%d/%y", errors="coerce")
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
```

```

df["Season"] = df["Month"].map({
    12: "Winter", 1: "Winter", 2: "Winter",
    3: "Spring", 4: "Spring", 5: "Spring",
    6: "Summer", 7: "Summer", 8: "Summer",
    9: "Fall", 10: "Fall", 11: "Fall",
})

# Time of day
def parse_hour(t):
    try:
        return int(str(t).split(":")[0])
    except Exception:
        return np.nan

df["Hour"] = df["Time"].apply(parse_hour)
df["TimeOfDay"] = pd.cut(
    df["Hour"],
    bins=[-1, 6, 12, 17, 20, 24],
    labels=["Night", "Morning", "Afternoon", "Evening", "Night2"],
)
df["TimeOfDay"] = df["TimeOfDay"].astype(str).replace("Night2", "Night")

# Depth
def parse_depth(d):
    try:
        return float(d)
    except Exception:
        return 0.0 # 'surface' or 'Unknown' → 0

df["DepthNum"] = df["Depth"].apply(parse_depth)
df["IsSubmerged"] = (df["DepthNum"] > 0).astype(int)

# Drop rows where target is missing
df = df.dropna(subset=["Injury"])
print(f"Working dataset: {len(df)} incidents\n")

```

Working dataset: 123 incidents

```

[6]: FEATURES = ["Mode", "Species", "County", "Season", "TimeOfDay",
                "IsSubmerged", "DepthNum", "Month", "Year"]
TARGET      = "Injury"

# Fill remaining NAs with 'Unknown'
for col in ["Mode", "Species", "County", "Season", "TimeOfDay"]:
    df[col] = df[col].fillna("Unknown").astype(str)

```

```

# Label-encode categorical features
le_dict = {}
X = df[FEATURES].copy()
for col in ["Mode", "Species", "County", "Season", "TimeOfDay"]:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    le_dict[col] = le

y_le = LabelEncoder()
y = y_le.fit_transform(df[TARGET])
class_names = y_le.classes_          # ['Fatal', 'Major', 'Minor']

print("Class distribution:")
for cls, cnt in zip(class_names, np.bincount(y)):
    print(f"  {cls:8s}: {cnt}")
print()

```

Class distribution:

```

Fatal    : 15
Major    : 59
Minor    : 49

```

```

[7]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=42, stratify=y
    )

rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    min_samples_split=2,
    class_weight="balanced",
    random_state=42,
)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(rf, X, y, cv=cv, scoring="accuracy")

print("=" * 50)
print("MODEL PERFORMANCE")
print("=" * 50)
print(f"Test accuracy      : {(y_pred == y_test).mean():.3f}")
print(f"CV accuracy (5-fold): {cv_scores.mean():.3f} ± {cv_scores.std():.3f}")

```

```
print()
print(classification_report(y_test, y_pred, target_names=class_names))
```

```
=====
MODEL PERFORMANCE
=====
```

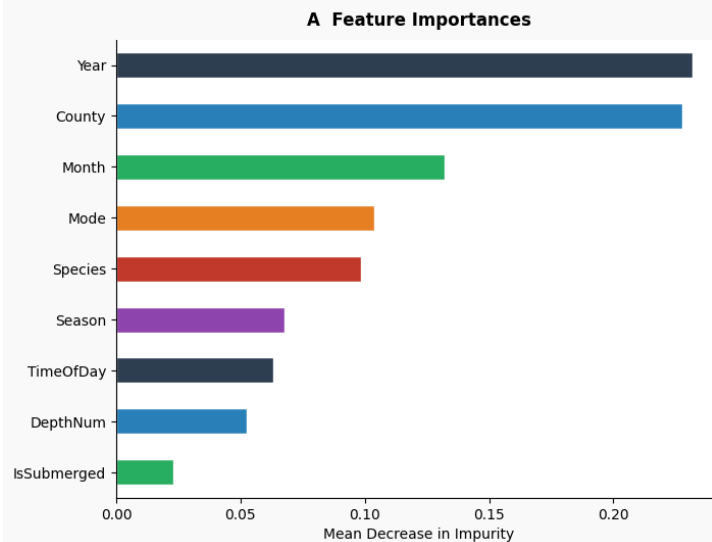
```
Test accuracy      : 0.640
CV accuracy (5-fold): 0.577 ± 0.069
```

	precision	recall	f1-score	support
Fatal	0.33	0.33	0.33	3
Major	0.69	0.75	0.72	12
Minor	0.67	0.60	0.63	10
accuracy			0.64	25
macro avg	0.56	0.56	0.56	25
weighted avg	0.64	0.64	0.64	25

```
[8]: fig = plt.figure(figsize=(18, 14))
fig.suptitle(
    "Shark Attack Severity - Random Forest Analysis\nCalifornia Incidents 1950-
    2022",
    fontsize=16, fontweight="bold", y=0.98
)
gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.4, wspace=0.35)

# Panel A: Feature Importance
ax1 = fig.add_subplot(gs[0, 0])
feat_imp = pd.Series(rf.feature_importances_, index=FEATURES).sort_values()
colors_bar = [COLORS[i % len(COLORS)] for i in range(len(feat_imp))]
feat_imp.plot(kind="barh", ax=ax1, color=colors_bar[:-1], edgecolor="white")
ax1.set_title("A Feature Importances", fontweight="bold", pad=10)
ax1.set_xlabel("Mean Decrease in Impurity")
for spine in ["top", "right"]:
    ax1.spines[spine].set_visible(False)
```

Shark Attack Severity — Random Forest Analysis California Incidents 1950-2022



```
[15]: fig2, axes = plt.subplots(1, 2, figsize=(14, 5))
fig2.suptitle("Additional Exploratory Breakdowns", fontsize=13,
             fontweight="bold")

# Species
species_injury = (
    df.groupby(["Species", "Injury"])
      .size().unstack(fill_value=0)
      .reindex(columns=["Minor", "Major", "Fatal"])
)
species_injury.plot(
    kind="bar", ax=axes[0], stacked=True,
    color=[PALETTE["Minor"], PALETTE["Major"], PALETTE["Fatal"]],
    edgecolor="white"
)
axes[0].set_title("Injury by Shark Species", fontweight="bold")
axes[0].tick_params(axis="x", rotation=20)
axes[0].set_ylabel("Incidents")
for sp in ["top", "right"]: axes[0].spines[sp].set_visible(False)

# Season
season_injury = (
    df.groupby(["Season", "Injury"])
      .size().unstack(fill_value=0)
      .reindex(columns=["Minor", "Major", "Fatal"])
)
```

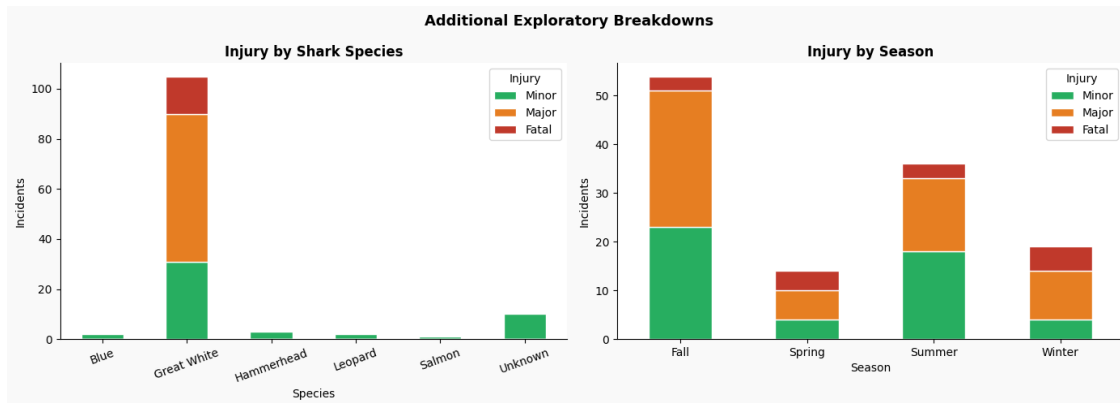
```

)
season_injury.plot(
    kind="bar", ax=axes[1], stacked=True,
    color=[PALETTE["Minor"], PALETTE["Major"], PALETTE["Fatal"]],
    edgecolor="white"
)
axes[1].set_title("Injury by Season", fontweight="bold")
axes[1].tick_params(axis="x", rotation=0)
axes[1].set_ylabel("Incidents")
for sp in ["top", "right"]: axes[1].spines[sp].set_visible(False)

plt.tight_layout()
plt.savefig("shark_rf_breakdowns.png",
            dpi=150, bbox_inches="tight")
print("Figure saved → shark_rf_breakdowns.png")

```

Figure saved → shark_rf_breakdowns.png



[]: